TBarSim: A Physics-Grounded Memristive 3D In-Memory Computing Simulator

Adib Kadir*
University of Texas at Arlington
Arlington, Texas, USA

Thongum Athoiba†
University of Texas at Arlington
Arlington, Texas, USA

Muhammad Rashed‡
University of Texas at Arlington
Arlington, Texas, USA

ACM Reference Format:
Adib Kadir, Thongum Athoiba, and Muhammad Rashed. 2026. TBarSim: A Physics-Grounded Memristive 3D In-Memory Computing Simulator. In . ACM, New York, NY, USA, 2 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Problem and Motivation

Modern AI workloads, such as CNNs and DNNs [1], suffer greatly from the limitations of the traditional von Neumann architecture, exposing the memory wall: the continuous shuttling of data between processing units and memory, a bottleneck widely acknowledged in the development of machine-learning accelerators like DianNao [2]. Processing-in-memory (PIM) directly addresses this bottleneck, and state-of-the-art PIM schemes have demonstrated strong results using 2D memristive crossbar arrays. Stacking these arrays into 3D architectures offers a natural path to further density, but introduces intricate parasitics and complex inter-layer current flows that 2D-focused tools and analyses do not capture. Existing simulation frameworks lack the automated capability to generate and solve the resulting nodal equations at 3D scale, leaving emerging 3D non-volatile memory technologies, such as ReRAM and MRAM, without the simulation support needed to realize their full potential.

2 Background and Related Work

To combat this, in-memory computing leverages the memristive crossbar array as a dominant hardware primitive. A crossbar utilizes a grid layout of memristors and wire segments to exploit Kirchhoff's Current Law (KCL) along bitlines, Kirchhoff's Voltage Law (KVL) along wordlines, and Ohm's law across memristors, performing analog computation natively. 2D crossbars have driven accelerators such as ISAAC [4] and PRIME [3], supported by simulators like CrossSim [8] and XBarSim [5].

Stacking crossbars with vertical interconnects offers a natural path to higher density and new accelerator topologies, but introduces vertical sneak paths and vertical voltage drops absent from any 2D layout, which completely alter the current distribution through the array. General-purpose SPICE simulators like Ngspice [7] can resolve these effects, but their general-purpose solver machinery and hand-written netlist workflows are not well suited to interactively exploring novel 3D topologies.

3 Novelty of the Approach

TBarSim closes that gap by taking inspiration from XBarSim's [5] node placement and KCL formulation and extending them to the 3D TBar architecture, formed by stacking crossbar arrays.

We formulate the TBar topology as an undirected weighted graph (V, E, w), with vertices corresponding to wordline, bitline, input, and output nodes, and edges corresponding to memristors, wire segments, and boundary resistors. For an $m \times n \times p$ architecture, this yields $2mnp + np + mp$ vertices. Applying Kirchhoff's Current Law across every non-trivial node produces a consistent linear system, which we assemble into a sparse $2mnp \times 2mnp$ conductance matrix and solve as $I = VG$. Rather than traverse the graph at runtime, we derive a closed-form indexing scheme covering 18 boundary-node categories for direct matrix construction; the graph itself is retained only for visualization.

We integrated the VTEAM voltage-controlled memristor model as a configurable option alongside static resistances. The simulator is delivered as a Python package with a real-time visualization tool built on this graph formulation (Figure 1), with core solve routines implemented in a dedicated C++ backend for performance.

4 Preliminary or Final Results

We validated TBarSim against Ngspice using two benchmark schemes swept across TBar dimensions.

Scheme A (VTEAM) models n^3 VTEAM memristors per $n \times n \times n$ architecture, initialized to alternating low- and high-resistance states with realistic device parameters. First-layer input terminals are driven by a time-varying signal encoding a 32-bit binary sequence, with all other terminals held at 0V, benchmarked from $n = 1$ to $n = 12$.

Scheme B (Fixed resistance) uses the same alternating low-/high-resistance initialization as a static linear circuit, with all input terminals held at a constant bias and all outputs grounded. We benchmarked this scheme from $n = 1$ to $n = 32$.

Across both schemes, TBarSim's computed nodal voltages and currents matched Ngspice's results to within 10^{-8}, confirming solver correctness, while achieving substantial speedup over Ngspice that widens with array size (Figure 2). This speedup stems from replacing general-purpose LU-based solving with a dedicated solver exploiting our KCL matrix's provable Symmetric Positive-Definite (SPD) structure.

*Project Representative, Undergraduate Student.
†Undergraduate Student.
‡Faculty Advisor.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, Washington, DC, USA
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

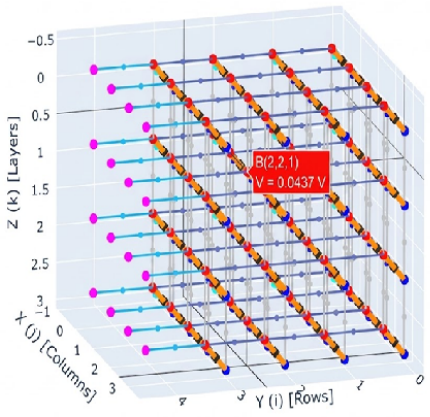


Figure 1: Real-time architectural rendering generated within the TBarSim environment. Unlike traditional command-line SPICE tools, TBarSim provides a unique, built-in graphical visualization of the memristive topology, allowing researchers to intuitively verify complex nodal connections and via placements.

Integrating VTEAM introduced a self-referential dependency between conductance and its rate of change, which most SPICE tools resolve via Jacobian-based Newton-Raphson iteration. Since this breaks the symmetric structure our solver depends on, we instead employ a Gummel iteration scheme, preserving SPD structure and solver speed while converging on the nonlinear device behavior.

5 Contributions and Significance

TBarSim's explicit 3D KCL solve provides exact inter-layer voltage visibility that macro-scale simulators lack, at speeds significantly faster than general-purpose SPICE tools.

In subsequent iterations, we plan to integrate additional nonlinear device models such as the Stanford RRAM model [6]. Because continuous-time SPICE solvers face convergence difficulty when evaluating memristive state derivatives ($\frac{dw}{dt}$) at scale – forcing adaptive time steps into the microsecond range and triggering Newton-Raphson bottlenecks – we are developing a discrete-time, cycle-accurate C++ backend that updates device conductances via discretized difference equations at fixed clock cycles. This preserves the symmetry of the KCL matrix at every step, allowing TBarSim to simulate dynamic AI inference workloads without sacrificing its SPD-based computational speedup.

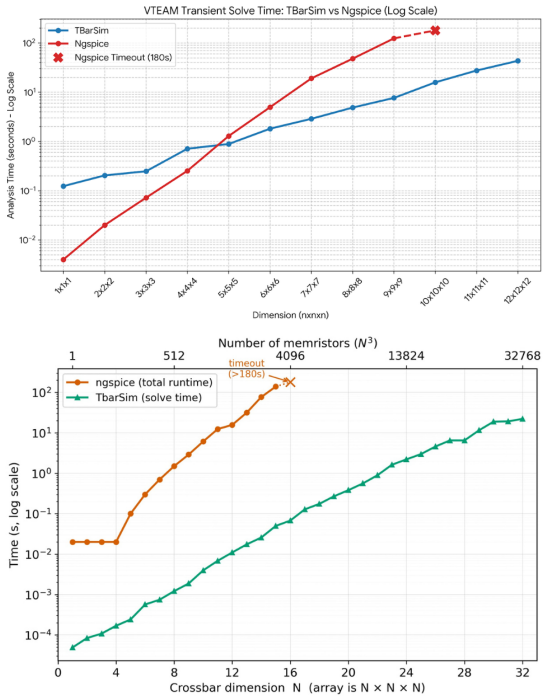


Figure 2: TBarSim vs. Ngspice solve time across TBar array sizes (log scale). (a) Transient solve time for the voltage-controlled VTEAM memristor model, swept from 1×1×1 to 12×12×12. TBarSim maintains near-flat solve time at small scale and overtakes Ngspice past 5×5×5; Ngspice exceeds a 180-second timeout by 10×10×10, while TBarSim continues scaling without a comparable transient bottleneck. (b) Total solve time for the linear (static-resistor) benchmark, swept from 1×1×1 up to 32×32×32 (32,768 memristors). Ngspice's runtime exceeds the same 180-second timeout near N=16; TBarSim continues scaling smoothly through N=32, several orders of magnitude faster at the largest size tested. Both benchmarks were run in Ngspice.

References

[1] AI Index Steering Committee. 2026. *The Artificial Intelligence Index 2026 Annual Report.* Technical Report. Institute for Human-Centered AI (HAI), Stanford University, Stanford, CA. https://hai.stanford.edu/assets/files/ai_index_report_2026.pdf

[2] Tianshi Chen, Zidong Du, Ninghui Sun, Jia Wang, Chengyong Wu, Yunji Chen, and Olivier Temam. 2014. DianNao: A Small-Footprint High-Throughput Accelerator for Ubiquitous Machine-Learning. In *Proceedings of the 19th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '14).* ACM, New York, NY, USA. doi:10.1145/2541940.2541967

[3] Ping Chi, Shuangchen Li, Zhenyu Qi, Peng Gu, Cong Xu, Tao Zhang, Jishen Zhao, Yongpan Liu, Yuan Wang, and Yuan Xie. 2016. PRIME: A Novel Processing-in-Memory Architecture for Neural Network Computation in ReRAM-Based Main Memory. In *Proceedings of the 43rd Annual International Symposium on Computer Architecture (ISCA '16).*

[4] Ali Shafiee, Anirban Nag, Naveen Muralimanohar, Rajeev Balasubramonian, John Paul Strachan, Miao Hu, R. Stanley Williams, and Vivek Srikumar. 2016. ISAAC: A Convolutional Neural Network Accelerator with In-Situ Analog Arithmetic in Crossbars. *ACM SIGARCH Computer Architecture News* 44, 3 (2016).

[5] Suyash Vardhan Singh, Anzhelika Kolinko, Md Hasibul Amin, Ramtin Zand, and Jason D. Bakos. 2025. A Decomposition-Based Memristive Crossbar Solver and FPGA-Accelerated Hardware Implementation. In *Proceedings of the Great Lakes Symposium on VLSI 2025 (GLSVLSI '25).* Association for Computing Machinery, New York, NY, USA, 581–586. doi:10.1145/3716368.3735282

[6] Stanford Nanoelectronics Lab. 2026. Stanford RRAM Model. https://nano.stanford.edu/downloads/stanford-rram-model. Accessed: 2026-07-21.

[7] Holger Vogt, Giles Atkinson, Dietmar Warning, and Paolo Nenzi. 2026. *Ngspice User's Manual Version 46.* Ngspice Project. https://ngspice.sourceforge.io/docs/ngspice-manual.pdf

[8] T. Patrick Xiao, Christopher H. Bennett, Ben Feinberg, Matthew J. Marinella, and Sapan Agarwal. 2022. *CrossSim Inference Manual v2.0.* Technical Report SAND2022-7074R. Sandia National Laboratories. doi:10.2172/1869509